Final Software Design Document (SDD)

1. Introduction & Scope

1.1 Purpose

Project Bridge is a high-trust, structured collaboration platform engineered to connect professionals across varying disciplines. The core problem it solves is moving beyond generic professional networking by providing a dedicated workspace where individuals can broadcast highly structured project ideas (domains, goals, stages) and initiate "focused rooms" to drive actionable collaboration. The target audience encompasses researchers, engineers, entrepreneurs, and creatives seeking verified partners.

1.2 Scope

System Boundaries: The system explicitly handles:

- User registration and authentication (with encrypted credentials).
- Public professional profile rendering.
- Project idea broadcasting (Project Board).
- Bidirectional network connection establishment (Friendships).
- Project-specific, threaded text-based messaging (Conversations).

Out of Scope: The system explicitly does *not* handle:

- Real-time bidirectional communication via WebSockets (the architecture assumes client-side fetching).
- Complex media or file uploads (avatars/attachments are out of scope).
- External OAuth or Social Single Sign-On (SSO).
- Horizontal scaling mechanisms out of the box, as it relies on a local JSON file.

2. System Architecture (In-Depth)

2.1 Architectural Strategy

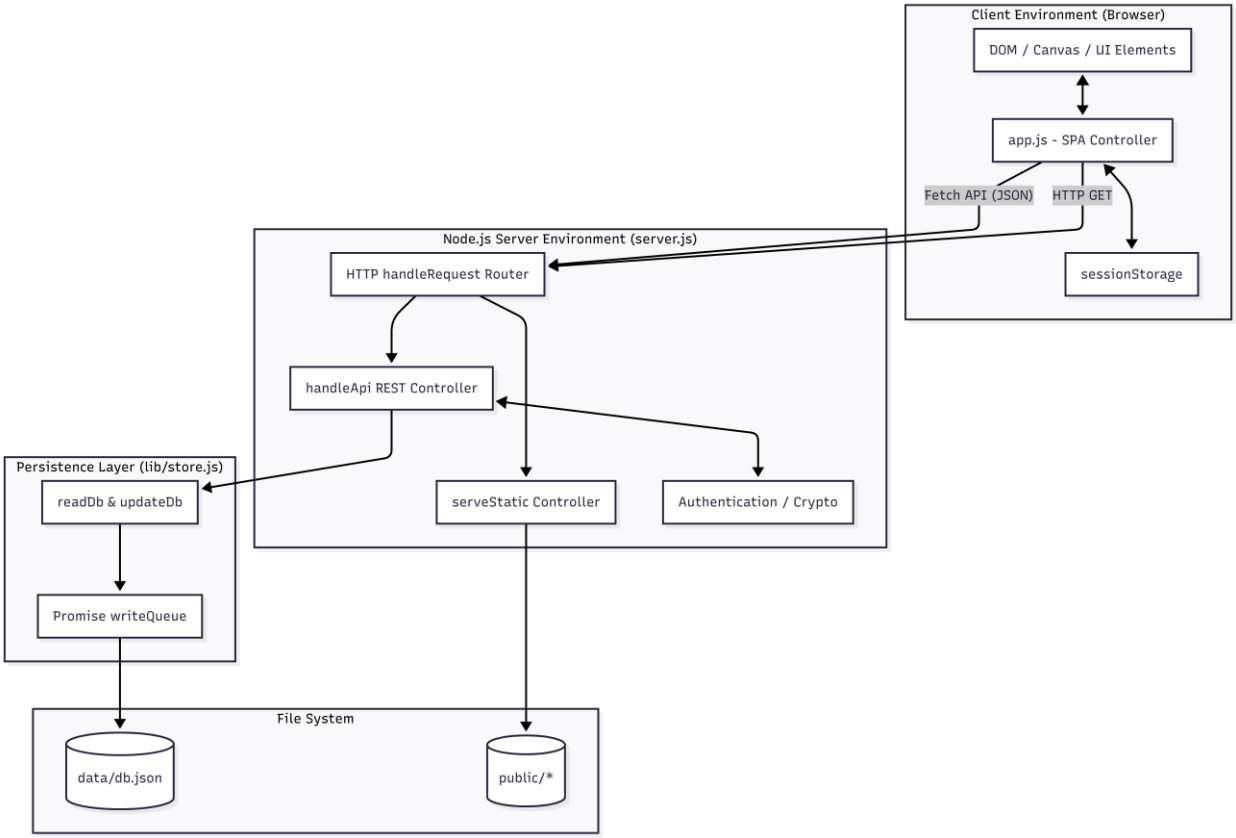
The project follows a **Monolithic Single Page Application (SPA)** architectural pattern coupled with a **Client-Server** backend model.

- **Proof in Codebase:** The entire application state and routing logic reside in a single `handleRequest` multiplexer within `server.js`, which bifurcates requests into either the `handleApi` REST router or the `serveStatic` file server. The application implements a minimal **Data Access Object (DAO)** pattern in `lib/store.js` to decouple flat-file disk persistence from API business logic.

2.2 Technology Stack

- **Frontend Interface:** HTML5, Vanilla CSS3 (`public/styles.css`), Vanilla JavaScript ES6+ (`public/app.js`).
    - *Architectural Deduction:* Chosen to achieve a near-zero memory footprint, zero-build-step deployment, and sub-second Initial Contentful Paint (ICP) without the computational overhead of Virtual DOM frameworks like React.
- **Backend Application Server:** Node.js (v18+ locally, `node:22-alpine` in Docker).
    - *Architectural Deduction:* The backend is built strictly using Node.js native modules (`http`, `path`, `fs/promises`, `crypto`). The `package.json` contains zero dependencies. This design eliminates third-party supply chain attack vectors and drastically reduces application boot time.
- **Data Persistence:** Flat-file JSON database (`data/db.json`).

2.3 System Architecture Diagram



3. Data Design & Layer

3.1 Data Models

Table 1: User Entity

Field	Data Type	Description
`id`	String	Unique identifier prefixed with u_ (e.g., `u_1j9f_xyz`).
`name`	String	Full user name concatenated from registration.
`role`	String	Professional role/title.
`specialty`	String	High-level specialty statement.
`bio`	String	Long-form biographical data.

`friendIds`	Array[String]	Array of foreign keys mapping to connected User IDs.

Table 2: Account Entity

Field	Data Type	Description
`id`	String	Unique identifier prefixed with `acc_`.
`userId`	String	Foreign key linking to the `User` entity.
`email`	String	Normalized user email address for authentication.
`passwordHash`	String	Cryptographically secured string (`salt:hash`).

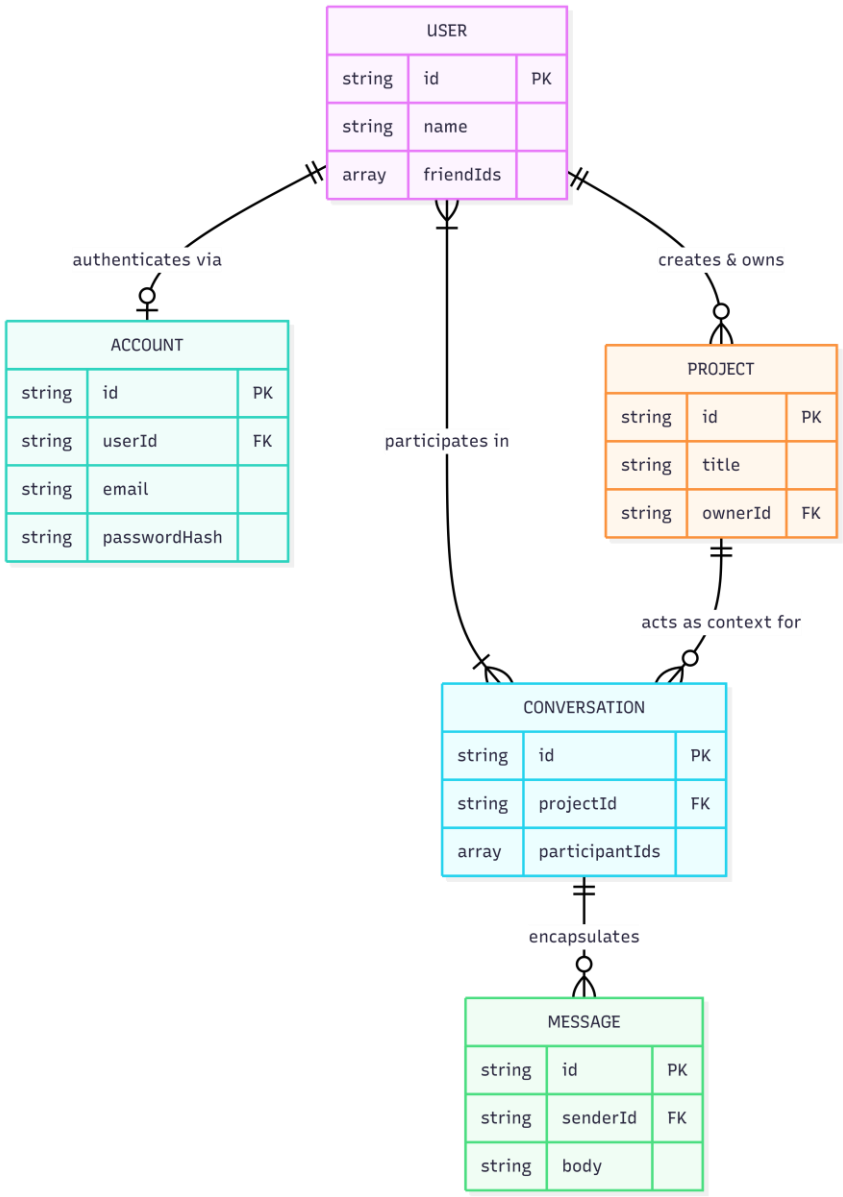
Table 3: Project Entity

Field	Data Type	Description
`id`	String	Unique identifier prefixed with `p_`.
`title`	String	Headline of the project.
`domain`	String	Industry category (e.g., Digital Health).
`ownerId`	String	Foreign key linking to the `User` entity that owns the project.
`tags`	Array[String]	Categorical tags used for filtering.

Table 4: Conversation Entity

Field	Data Type	Description
`id`	String	Unique identifier prefixed with `c_`.
`projectId`	String	Foreign key linking to the discussed `Project`.
`participantIds`	Array[String]	Sorted array of exactly two `User` IDs representing the room members.
`messages`	Array[Object]	Embedded array of Message objects (`id`, `senderId`, `body`, `createdAt`).

3.2 Entity-Relationship (ER) Diagram



3.3 Data Storage Logic

Data is persistently stored on disk as a serialized JSON tree inside `data/db.json`. **Consistency Strategy:** Because Node.js handles multiple asynchronous requests, concurrent disk writes could corrupt the JSON file. The system prevents this by leveraging a sequential Promise chain located in `lib/store.js`:

```
let writeQueue = Promise.resolve();
```

Every mutation calls `updateDb(mutator)`. This appends a `.then()` to the global `writeQueue`. It awaits the completion of previous file writes, executes `readDb()` to

pull the absolute latest state into memory, applies the specific mutation, and finally utilizes `fs.writeFile` to flush the new state back to disk. This constitutes an atomic application-level lock per transaction.

4. Component & Module Details

4.1 Module Analysis

- **`/server.js`**: The HTTP application server. It mounts the Node native `http.createServer()` and defines `handleRequest(req, res)`. It encapsulates the raw REST implementations, body parsing, payload validations, and cryptographic logic.
- **`/lib/store.js`**: The Data Access Object (DAO). It establishes the environment-specific target path (`DATA_FILE`), manages directory creation if the file doesn't exist (`ensureDbFile`), and exports `readDb()` and `updateDb(mutator)`.
- **`/api/index.js`**: The Serverless Adapter. It imports `handleRequest` from `server.js` and exports it as an asynchronous function specifically typed to handle Vercel serverless HTTP invocations.
- **`/public/app.js`**: The Monolithic Client Controller. It houses the `state` management object, maps over 30 UI DOM nodes into the `elements` dictionary, calculates the `DNAHelix` canvas animation via RequestAnimationFrame loops, and intercepts all form submittals via event listeners to trigger asynchronous `request(url, options)` calls.

4.2 Critical Functions

1. **`hashPassword(password, salt = crypto.randomBytes(16).toString("hex"))` (server.js)**
    a. *Inputs:* Plaintext password, optional hexadecimal salt.
    b. *Logic:* Executes the `scrypt` key derivation function (`crypto.scryptSync(password, salt, 64)`) to securely hash the password against brute-force/rainbow table attacks.
    c. *Outputs:* Returns a combined string in the format `"salt:hash"`.
2. **`parseBody(req)` (server.js)**
    a. *Inputs:* Native Node.js `http.IncomingMessage` request stream.
    b. *Logic:* Iteratively attaches `'data'` chunks to a string. Enforces a strict 1MB size limit (`if (raw.length > 1_000_000)`), forcefully calling `req.destroy()` if exceeded to prevent RAM exhaustion attacks. Parses the string via `JSON.parse`.

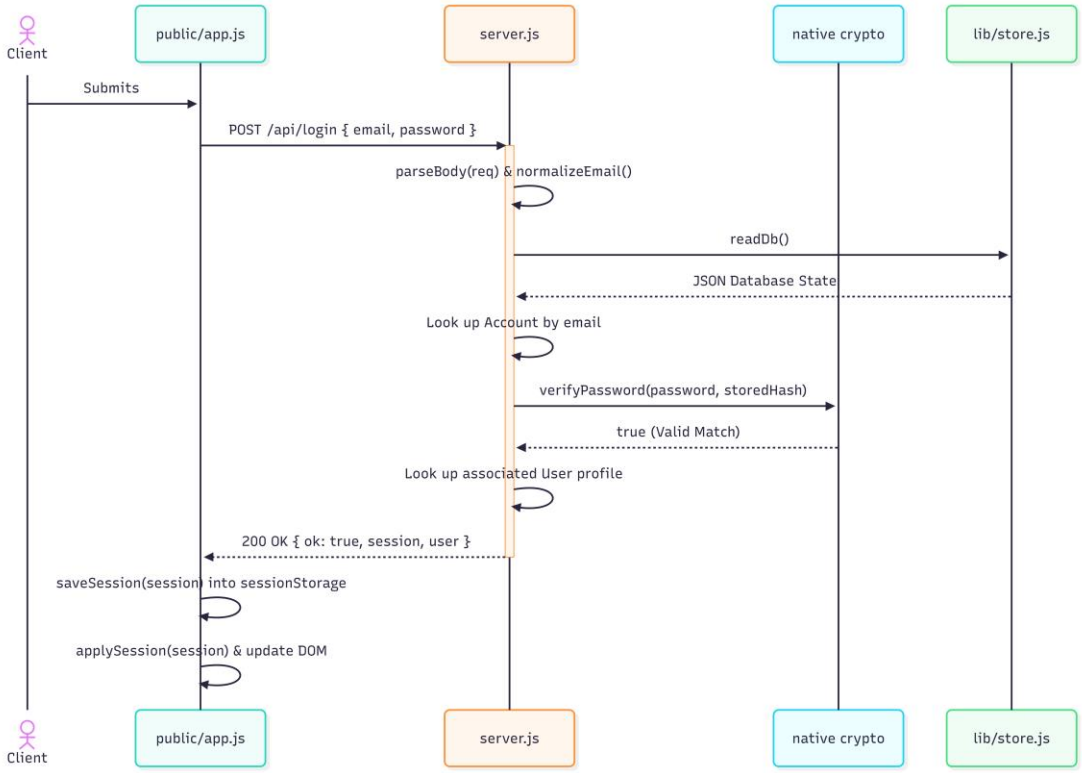
    c. *Outputs:* Returns `Promise<Object>`.
3. **`serveStatic(req, res, pathname)` (server.js)**
    a. *Inputs:* HTTP Request, HTTP Response, requested URL pathname.
    b. *Logic:* Secures the path via `path.normalize(requestedPath).replace(/^(\.\.[/\\])+/, "")`. It explicitly checks `if (!filePath.startsWith(PUBLIC_DIR))` to block path traversal. Automatically resolves `/` or directory paths to `index.html`. Dynamically attaches MIME types based on file extensions.
    c. *Outputs:* Pipes the file buffer to `res.end(file)`.
4. **`updateDb(mutator)` (lib/store.js)**
    a. *Inputs:* A callback function `mutator(db)`.
    b. *Logic:* Chains onto `writeQueue`. Reads the DB, passes it to the mutator. If mutator executes without throwing, the entire DB object is stringified and written to `DATA_FILE`.
    c. *Outputs:* Returns a Promise resolving to the exact entity created/modified by the mutator.

4.3 API Reference

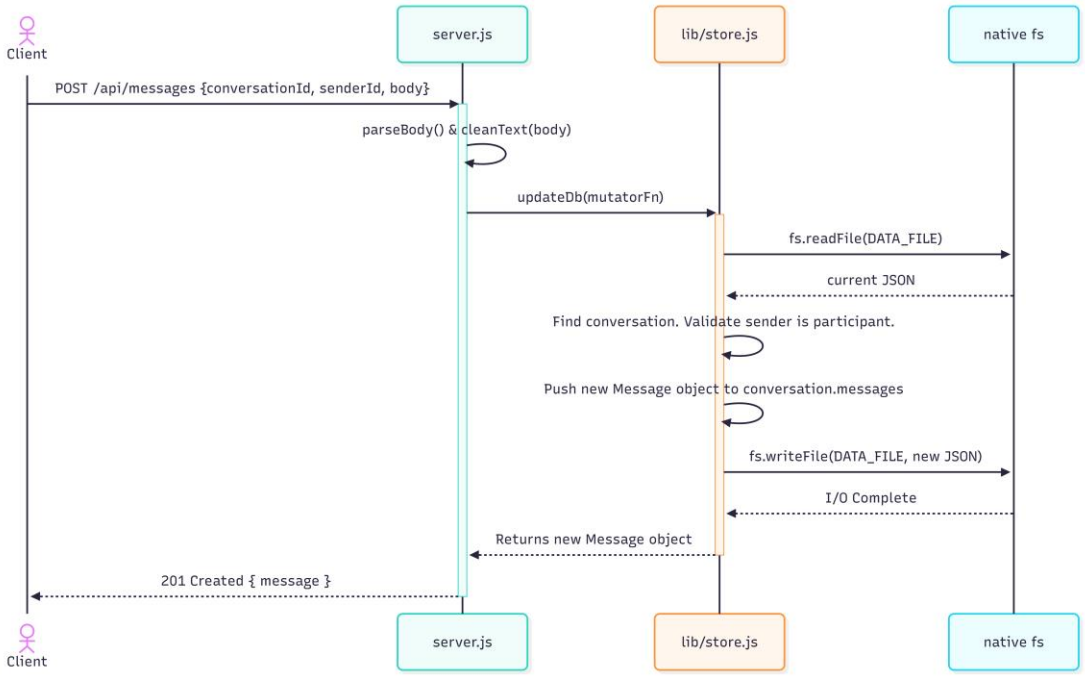
- **`GET /api/bootstrap`**
    - *Response (200):* `{ users: Array, projects: Array, conversations: Array, stats: Object }`
- **`POST /api/register`**
    - *Payload:* `{ firstName, lastName, email, phone, profession, password }`
    - *Response (201):* `{ ok: true, user: Object }` | *Error (400):* `{ error: "An account with this email already exists" }`
- **`POST /api/login`**
    - *Payload:* `{ email, password }`
    - *Response (200):* `{ ok: true, session: { userId, email }, user: Object }` | *Error (401):* `{ error: "Incorrect email or password" }`
- **`POST /api/messages`**
    - *Payload:* `{ conversationId, senderId, body }`
    - *Response (201):* `{ message: Object }`

5. Workflows (Sequence Diagrams)

5.1 Workflow: User Authentication (Login)



5.2 Workflow: Sending a Message to a Room



6. Non-Functional Requirements & Security

6.1 Security

- **Cryptographic Hashing**: The system enforces password protection using the Memory-Hard `scrypt` Key Derivation Function (`crypto.scryptSync(password, salt, 64)`) to actively thwart GPU-accelerated dictionary attacks.
- **Path Traversal Prevention**: Within `serveStatic`, paths are rigidly sanitized. `path.normalize(requestedPath).replace(/^(\.\.[/\\])+/, "")` strips relative jump operators, followed by a hard assertion `if (!filePath.startsWith(PUBLIC_DIR))` guaranteeing access is jailed to the `public/` directory.
- **Cross-Site Scripting (XSS) Mitigation**: The backend operates strictly via JSON payloads, implicitly preventing arbitrary code execution. The frontend forces all DOM injections through `escapeHtml(value)` in `app.js`, replacing `<, >, &, ", '` with strict HTML entities.
- **Payload Capping**: `parseBody(req)` intercepts streams exceeding 1,000,000 bytes, severing the TCP connection (`req.destroy()`) to prevent Buffer Overflows and Application-Layer DDoS attacks.

6.2 Error Handling

- **Global Catch Blocks**: The `server.js` multiplexer (`handleRequest`) wraps routing inside a `try...catch` block. Unhandled exceptions are serialized securely to the client via a centralized 500 status responder: `json(res, 500, { error: "Server error", details: error.message })`.
- **Client Normalization**: The `normalizeRequestError(error)` function in `app.js` catches fetch rejections, intelligently interpreting `"failed to fetch"` or `"networkerror"` as the standardized UI message: `APP_SERVER_UNREACHABLE_MESSAGE`.

6.3 Performance

- **Memory Footprint**: By omitting heavy middleware like Express.js, the backend boots almost instantly and idles at < 30MB of RAM.
- **Caching Strategy**: `res.setHeader("Cache-Control", "no-store")` is systematically injected on every response to prevent browser caching of stale JSON payloads, prioritizing data recency over bandwidth.

7. Deployment & Environment

7.1 Environment Variables

Variable	Required	Description
`PORT`	Optional	Binds the Node.js server to a specific port. Defaults to `3000`.
`NODE_ENV`	Optional	Context for execution. Used predominantly in Docker as `production`.
`HOST_PORT`	Optional	Defined in `.env` for Docker Compose. Maps the external host port to internal port 3000 (Defaults to `3000` via string interpolation `${HOST_PORT:-3000}`).

7.2 Deployment Strategy

1. **Local Execution**: Exclusively requires Node.js v18+. Execution commands: `npm install` (empty resolution) followed by `npm start` or `npm run dev`.

2. **Dockerized Architecture**: The included `Dockerfile` utilizes a multi-stage approach, resulting in an exceptionally slim container (`node:22-alpine`). Via `docker compose up --build -d`, the application mounts the local `./data` directory dynamically (`./data:/app/data`), guaranteeing zero data loss during container restarts.
3. **Vercel / Serverless**: The infrastructure provides a Vercel routing configuration (`vercel.json`). The entire `server.js` router is exported and consumed by `api/index.js`, adapting the monolithic HTTP server to the Vercel AWS Lambda signature effortlessly.

8. Architectural Assumptions & Recommendations

1. **Memory Boundary Assumption**: The current `lib/store.js` implementation loads the *entire* `db.json` string into memory during every read and write cycle.
    a. *Recommendation*: For production loads exceeding 5,000 users, migrate the persistence layer to an embedded SQLite database or PostgreSQL using Prisma ORM to allow indexed queries and cursor-based pagination.
2. **Session Security Assumption**: User context currently relies completely on a plaintext user object saved within the browser's `sessionStorage`.
    a. *Recommendation*: Implement a robust JSON Web Token (JWT) architecture, issuing stateless tokens inside `HttpOnly`, `Secure` cookies to completely eliminate local session hijacking risks.
3. **Real-time Communication Deficit**: The messaging interface in `conversations` relies entirely on manual DOM refreshing or polling to retrieve new messages.
    a. *Recommendation*: Implement WebSockets (`ws` module) to broadcast mutation events specifically when `POST /api/messages` successfully pushes to the `updateDb` queue, facilitating real-time UX.